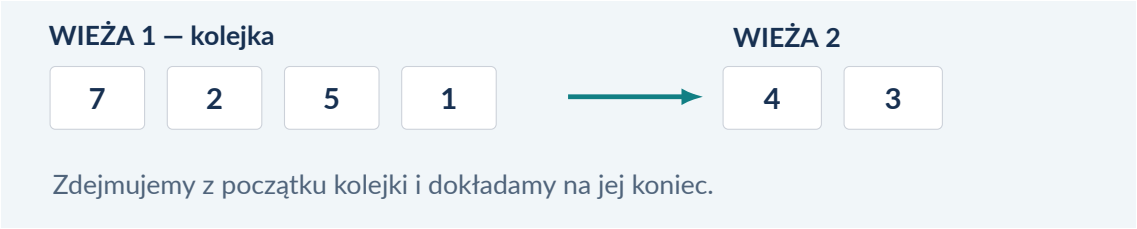
Od zera do rozwiązania
Hanoj kontratakuje

Algebra, algorytmy, kolejki i sortowanie bitowe



Poziom: od zupełnych podstaw · Język: polski · C++17

Materiał dydaktyczny na podstawie zadania „Hanoj kontratakuje” (HAK), XXXIV Olimpiada Informatyczna, etap I, 2026. Nie jest oficjalnym omówieniem organizatorów.

Spis treści

Jak korzystać z tego podręcznika 3

Lekcja 1. Algebra od samych podstaw 3
1.1. Liczby, zmienne i wyrażenia . . . 3
1.2. Suma i zapis z symbolem sigma . . . 3
1.3. Potęgi i liczby dwójkowe . . . 4
1.4. Logarytm — bez strachu . . . 4
1.5. Nierówności i szacowanie . . . 4
Sprawdź się: algebra . . . 5

Lekcja 2. Co to jest algorytm? 5
2.1. Wejście, wyjście, ograniczenia . . . 5
2.2. Trzy pytania przed napisaniem kodu . . . 5
2.3. Pseudokod . . . 5
2.4. Niezmiennik . . . 5

Lekcja 3. Wieża to kolejka FIFO 6
3.1. Dlaczego nie stos? . . . 6
3.2. Jak zapisać kolejkę w C++? . . . 6
3.3. Jak myśleć o legalności ruchu? . . . 6

Lekcja 4. Zrozum zadanie HAK 6
4.1. Opis prostymi słowami . . . 7
4.2. Nasz przykład przewodni . . . 7
4.3. Pomysł: najpierw uprość problem . . . 8

Lekcja 5. Nadaj krążkom rangi 8
5.1. Numer krążka nie mówi, gdzie ma trafić . . . 8
5.2. Dlaczego rang jest dokładnie n? . . . 8
5.3. Zebranie wszystkich krążków . . . 8

Lekcja 6. Stabilne rozdzielanie według jednego bitu 9
6.1. Co znaczy „stabilnie”? . . . 9
6.2. Jeden przebieg: dokładna symulacja . . . 9
6.3. Dlaczego wykonujemy dokładnie n kroków? . . . 10

Lekcja 7. Radix Sort: od najmłodszego bitu 10
7.1. Jeden bit nie wystarcza . . . 10
7.2. Pełny przykład z pięcioma rangami . . . 10
7.3. Rozdzielenie po sortowaniu . . . 10

Lekcja 8. Trzy fazy jako jeden algorytm 11

Lekcja 9. Dowód poprawności krok po kroku 11
9.1. Co to jest dowód poprawności? . . . 11
9.2. Lemat 1: poprawność pojedynczego rozdzielenia . . . 11
9.3. Lemat 2: poprawność sortowania . . . 12
9.4. Lemat 3: poprawność końcowego rozdzielenia . . . 12

Lekcja 10. Liczenie ruchów i pamięci 12
10.1. Ile ruchów ma pierwsza faza? . . . 12
10.2. Ile ruchów ma jeden przebieg? . . . 12

10.3. Trzecia faza i pełny wzór . . . 12
10.4. Dlaczego limit 4 milionów jest spełniony? . . . 13
10.5. Złożoność obliczeniowa . . . 13

Lekcja 11. Pełny program w C++17 **13**

Lekcja 12. Czytamy kod, zamiast go przepisywać **15**
12.1. `vector` i `deque` . . . 15
12.2. Pętle i warunki . . . 16
12.3. Wyciąganie pojedynczego bitu . . . 16
12.4. Funkcja wykonująca ruch . . . 16
12.5. Po co `long long` i `1LL`? . . . 16
12.6. Dlaczego program nie zapisuje wszystkich ruchów? . . . 16

Lekcja 13. Jak samodzielnie testować rozwiązania **17**
13.1. Kompilacja . . . 17
13.2. Testy brzegowe . . . 17
13.3. Symulator jako niezależny kontroler . . . 17
13.4. Weryfikacja tej implementacji . . . 17

Lekcja 14. Najczęstsze błędy i jak ich unikać **18**

Lekcja 15. Ćwiczenia do samodzielnego rozwiązania **18**
Poziom 1 — podstawy . . . 18
Poziom 2 — modelowanie i sortowanie . . . 18
Poziom 3 — dowodzenie i analiza . . . 19

Rozwiązania ćwiczeń **19**

Lekcja 16. Jak samodzielnie wymyślać rozwiązania podobnych zadań? **19**
Krok 1. Przetłumacz treść na operacje . . . 19
Krok 2. Oddziel wymagany wynik od sposobu jego osiągnięcia . . . 20
Krok 3. Zamień limit na budżet na element . . . 20
Krok 4. Poszukaj stanu pośredniego . . . 20
Krok 5. Zmień nazwy elementów, jeśli upraszczają problem . . . 20
Krok 6. Znajdź małą operację, którą możesz wielokrotnie powtarzać . . . 20
Krok 7. Zapisz niezmiennik i oszacuj koszt **przed** programowaniem . . . 20
Krok 8. Rozdziel generator od kontrolera . . . 20

Cztery tygodnie nauki od zera **21**
Co rozwiązywać dalej? . . . 21

Jedna strona do powtórki **21**

Jak korzystać z tego podręcznika

Nie musisz znać algebry, struktur danych ani języka C++, żeby zacząć. Materiał prowadzi od prostego przykładu z kartkami leżącymi na stosie do pełnego programu, który rozwiązuje zadanie olimpijskie.

Proponowana metoda: czytaj po jednej lekcji, a następnie **samodzielnie przeliczaj przykład na kartce**. Kod uruchom dopiero wtedy, kiedy rozumiesz, dlaczego każda operacja jest legalna. Zatrzymaj się przy zadaniach ćwiczeniowych; rozwiązania znajdują się na końcu.

Cele nauki:

1. Rozumiesz podstawowe zapisy matematyczne: zmienną, sumę, potęgę i nierówność.
2. Potrafisz rozpoznać kolejkę FIFO i wykonać na niej operacje.
3. Umiesz uprościć trudne zadanie, nadając elementom nowe numery.
4. Rozumiesz, czym jest stabilne sortowanie według bitów.
5. Potrafisz udowodnić poprawność i policzyć maksymalną liczbę operacji.
6. Czytasz pełne rozwiązanie w C++17 i wiesz, jak je przetestować.

> **Ważne:** ten problem nie wymaga znalezienia *najkrótszego* rozwiązania. Wymaga dowolnego rozwiązania o liczbie ruchów nieprzekraczającej 4 000 000. Ta różnica bardzo ułatwia projektowanie algorytmu.

Lekcja 1. Algebra od samych podstaw

1.1. Liczby, zmienne i wyrażenia

Liczba oznacza konkretną wartość, np. `5`. **Zmienna** to nazwa, pod którą przechowujemy pewną wartość. Jeśli piszemy $n = 5$, czytamy: „n jest równe pięć”. W zadaniach algorytmicznych n często oznacza liczbę elementów, a k liczbę pojemników, w tym przypadku wież.

Wyrażenie $2n + 1$ oznacza „dwa razy n, plus jeden”. Dla $n = 5$ otrzymujemy $2 \cdot 5 + 1 = 11$.

Nie myl trzech zapisów:

- $n = 5$ w matematyce: stwierdzenie o wartości zmiennej.
- `n = 5;` w C++: przypisz do zmiennej `n` wartość `5`.
- `n == 5` w C++: sprawdź, czy wartość `n` wynosi `5`.

Indeks wskazuje pozycję: c_1 to liczba krążków na pierwszej wieży, a c_2 na drugiej. Zapis c_i oznacza liczbę krążków na dowolnej wieży numer i.

1.2. Suma i zapis z symbolem sigma

Jeżeli mamy trzy wieże, a na nich odpowiednio 2, 1 i 3 krążki, to całkowita liczba krążków wynosi

$$n = c_1 + c_2 + c_3 = 2 + 1 + 3 = 6.$$

Matematycy często zapisują to krócej:

$$n = \sum_{i=1}^{k} c_i.$$

Symbol $\sum$ czytamy „suma”. Oznacza on: „dodawaj kolejne wartości c_i, zaczynając od $i = 1$, a kończąc na $i = k$”. Nie jest to nowa, magiczna operacja — jedynie skrót długiego dodawania.

Miniprzykład: jeżeli $c_1 = 4, c_2 = 0, c_3 = 2$, to $n = 4 + 0 + 2 = 6$. Pusta wieża jest dozwolona.

1.3. Potęgi i liczby dwójkowe

Potęga 2^3 oznacza $2 \cdot 2 \cdot 2 = 8$. Najważniejsze wartości dla tej pracy:

Potęga	Wartość	Potęga	Wartość
2^0	1	2^5	32
2^1	2	2^{10}	1 024
2^2	4	2^{16}	65 536
2^3	8	2^{17}	131 072
2^4	16	2^{20}	1 048 576

Komputer zwykle opisuje liczby za pomocą **bitów** — cyfr `0` i `1`. Zapis $\texttt{10110}_2$ oznacza:

$$1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 0 + 4 + 2 + 0 = 22.$$

W tym dokumencie bity liczymy od prawej strony: bit `0` to najmłodszy, bit `1` to następny, itd.

Liczba dziesiętna	Zapis binarny	Bit 0	Bit 1	Bit 2
0	`000`	0	0	0
1	`001`	1	0	0
2	`010`	0	1	0
3	`011`	1	1	0
4	`100`	0	0	1
5	`101`	1	0	1

1.4. Logarytm — bez strachu

Logarytm o podstawie 2 odpowiada na pytanie: „do jakiej potęgi podnieść 2, aby otrzymać daną liczbę?”. Na przykład $\log_2 8 = 3$, bo $2^3 = 8$.

W naszym algorytmie ważne jest, ile bitów wystarcza do opisania wszystkich rang od 0 do $n-1$. Dla $n \geq 2$ jest to

$$B = \lceil \log_2 n \rceil.$$

Nawiasy $\lceil\ \rceil$ oznaczają **zaokrąglenie w górę do liczby całkowitej**. Dla $n = 5$ mamy $\log_2 5 \approx 2{,}32$, więc $B = 3$.

W programie nie musimy liczyć logarytmu! Zamiast tego zwiększamy `bits`, dopóki $2^{\text{bits}} < n$. Jest to prostsze i unika problemów z dokładnością liczb rzeczywistych.

Dla $n = 100\,000$ mamy $2^{16} = 65\,536 < 100\,000 \leq 131\,072 = 2^{17}$, dlatego wystarcza $B = 17$. Dla $n = 1$ wystarcza $B = 0$: mamy tylko jedną możliwą rangę `0` i niczego nie trzeba sortować.

1.5. Nierówności i szacowanie

Zapis $x \leq 100$ znaczy: „x jest mniejsze lub równe 100”. Jeżeli udowodnimy, że

$$M \leq 36n \qquad \text{oraz} \qquad n \leq 100\,000,$$

to automatycznie wiemy, że $M \leq 3\,600\,000$. A ponieważ $3\,600\,000 < 4\,000\,000$, rozwiązanie mieści się w limicie.

W informatyce często nie musimy znać dokładnego wyniku. Wystarczy **bezpieczna górna granica**, prawdziwa dla każdego wejścia.

Sprawdź się: algebra

A1. Oblicz $3n + 2$ dla $n = 4$. **A2.** Ile wynosi 2^6? **A3.** Ile bitów potrzeba na rangi `0, 1, 2, 3, 4, 5`? **A4.** Zapisz `13` binarnie. Odpowiedzi są w rozdziale „Rozwiązania ćwiczeń”.

Lekcja 2. Co to jest algorytm?

Algorytm to precyzyjna instrukcja, która krok po kroku przekształca dane wejściowe w wymagany wynik. Ważne, by każdy krok dało się wykonać bez domysłów.

Przykład: „posortuj krążki” to jeszcze nie algorytm — nie wiemy, jakimi dozwolonymi ruchami to zrobić. Natomiast: „powtórz dokładnie `n` razy: odczytaj pierwszy krążek, sprawdź jego bit, przenieś do właściwej kolejki” — to fragment algorytmu.

2.1. Wejście, wyjście, ograniczenia

Zadanie zawiera trzy niezależne wymagania:

1. **Poprawność ruchu:** krążek można zabrać wyłącznie z góry niepustej wieży i umieścić na spodzie dowolnej wieży, również tej samej.
2. **Poprawność wyniku:** po wszystkich ruchach każda wieża musi wyglądać identycznie jak w konfiguracji pierwotnej, włącznie z kolejnością.
3. **Limit:** trzeba wypisać najwyżej 4 000 000 ruchów; pamięć dostępna programowi to 256 MB.

Nie ma warunku, by użyć najmniejszej możliwej liczby ruchów.

2.2. Trzy pytania przed napisaniem kodu

- **Co pozostaje niezmienne?** Zestaw krążków: żadnego nie tworzymy ani nie usuwamy.
- **Co możemy swobodnie zmieniać?** Przynależność krążka do wieży i kolejność w wieży.
- **Jaką operację oferuje zadanie?** Wyjęcie pierwszego elementu jednej kolejki i dopisanie go na koniec innej.

Ostatnia odpowiedź prowadzi nas do właściwej struktury danych.

2.3. Pseudokod

Pseudokod to przepis napisany zwykłym językiem, bez składni konkretnego języka programowania:

```
DOPÓKI wieża 2 nie jest pusta:
    weź pierwszy dysk z wieży 2
    dołóż go na koniec wieży 1
```

Dopiero potem przekładamy go na C++.

2.4. Niezmiennik

Niezmiennik to fakt, który pozostaje prawdziwy na początku i końcu każdego obrotu pętli. Na przykład: „po przetworzeniu `j` krążków ich wzajemna kolejność w obrębie odpowiedniej grupy jest taka sama jak na początku”.

Dowód poprawności można ułożyć jak domino:

1. Pokaż, że niezmiennik jest prawdziwy przed pierwszym krokiem.
2. Pokaż, że jeden krok go nie niszczy.
3. Wywnioskuj, co jest prawdziwe po ostatnim kroku.

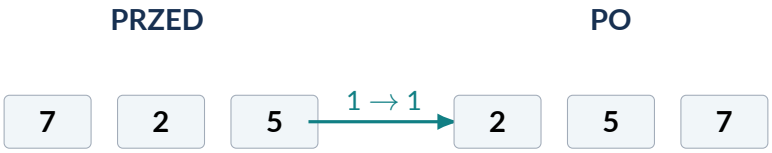
Ta metoda jest niezbędna w zadaniach olimpijskich.

Lekcja 3. Wieża to kolejka FIFO

3.1. Dlaczego nie stos?

Na pierwszy rzut oka wieża przypomina stos, bo zabieramy element z góry. Zwykły **stos LIFO** ma jednak operacje „zdejmij z góry” oraz „dołóż na górę”. Tutaj dokładamy na **dół**, więc właściwą abstrakcją jest **kolejka FIFO**: pierwszy element wchodzi, pierwszy wychodzi.

Zapis `[7, 2, 5]` czytamy od góry do dołu: `7` jest na początku kolejki.



Ruch z wieży na tę samą wieżę to obrót kolejki.

Ruch `1 → 1` zmienia `[7, 2, 5]` w `[2, 5, 7]`. **Nie jest operacją pustą**: obraca kolejkę o jedno miejsce. Ta własność jest kluczowa.

3.2. Jak zapisać kolejkę w C++?

Do tego zadania wygodny jest `deque<int>`: można odczytać i usunąć element z początku, a także dodać element na koniec.

```cpp
#include <deque>
using namespace std;

deque<int> q = {7, 2, 5};
int x = q.front();  // x = 7
q.pop_front();      // q = [2, 5]
q.push_back(x);     // q = [2, 5, 7]
```

W naszym rozwiązaniu tablica `towers` przechowuje `k` takich kolejek.

3.3. Jak myśleć o legalności ruchu?

Jeśli `towers[a]` nie jest pusta, to:

```
disk = pierwszy element towers[a]
usuń go z początku towers[a]
dodaj go na koniec towers[b]
wypisz "a b"
```

Tutaj `a` i `b` to numery wież. Gdy `a == b`, ta sama instrukcja nadal działa poprawnie.

Ćwiczenie K1: zapisz, jak wygląda kolejka `[4, 1, 3, 2]` po dwóch ruchach `1 → 1`. **K2:** czy w ruchu `1 → 2` możesz przenieść drugi od góry dysk, nie ruszając pierwszego?

Lekcja 4. Zrozum zadanie HAK

4.1. Opis prostymi słowami

Mamy k wież i n rozróżnialnych krążków. Znamy dwa stany: **docelowy**, który chcemy odtworzyć, oraz **aktualny**, od którego zaczynamy. Kolejności na wieżach mogą być dowolne; rozmiar krążka nie wpływa na możliwość ruchu.

W zadaniu: $2 \le k \le 5$, $1 \le n \le 100\,000$, a limit wynosi 4 000 000 operacji. Dane opisują każdą wieżę od **góry do dołu**.

Na wejściu najpierw przychodzi konfiguracja **docelowa**, potem **aktualna**. To dość częste źródło błędów.

4.2. Nasz przykład przewodni

Przyjmijmy $k = 3$ i $n = 5$.

Wieża	Stan docelowy	Stan aktualny
1	`[4, 1]`	`[2, 4, 5]`
2	`[3]`	`[1]`
3	`[2, 5]`	`[3]`

Przykładowy plik wejściowy wygląda tak:

```
3
2 4 1
1 3
2 2 5
3 2 4 5
1 1
1 3
```

Liczba rozpoczynająca każdy wiersz wieży mówi, ile krążków opisano dalej w tym wierszu. Dla pustej wieży cały wiersz to `0`.

Format wyjścia: w pierwszej linii wypisz liczbę ruchów `m`. W kolejnych `m` liniach wypisz po dwie liczby `a b`: numer wieży źródłowej i numer wieży docelowej. Numery należą do przedziału `1..k`, wieża źródłowa nie może być pusta, a `m` nie może przekroczyć 4 000 000.

Dla powyższego wejścia nasz program wypisze na przykład poniższe **27** ruchów:

```
27
2 1
3 1
1 2
1 1
1 1
1 2
1 1
2 1
2 1
1 1
1 1
1 2
1 2
1 1
2 1
2 1
1 1
1 2
```

```
1 1
1 1
1 1
2 1
1 1
1 1
1 2
1 3
1 3
```

Widzisz, że na wyjściu występują także ruchy `1 1`: są legalne i faktycznie obracają pierwszą kolejkę. Nie trzeba otrzymać dokładnie tego samego zestawu ruchów co tutaj; sprawdzana jest ich legalność oraz końcowy stan.

4.3. Pomysł: najpierw uprość problem

Nie musimy „odwijać" ruchów Bajtka. Nie wiemy nawet, jakie ruchy wykonał, i nie musimy tego odtwarzać.

Zamiast tego:

1. Zbierzemy wszystkie krążki na wieży 1.
2. Posortujemy je w kolejności, w jakiej powinny występować w konfiguracji docelowej.
3. Z rozsortowanej kolejki rozdysponujemy je na właściwe wieże.

Do sortowania wystarczą wieże **1 i 2**. Pozostałe mogą w tym czasie pozostawać puste.

Lekcja 5. Nadaj krążkom rangi

5.1. Numer krążka nie mówi, gdzie ma trafić

Numer `1` nie oznacza, że krążek ma znaleźć się na pierwszej pozycji. W naszym celu pierwszy od góry na pierwszej wieży jest krążek `4`.

Wprowadzamy więc **rangę docelową**: kolejną pozycję w całkowitym porządku, wyznaczonym przez odczytanie docelowej wieży 1 od góry do dołu, potem wieży 2 itd.

Kolejność docelowa	Krążek	Ranga	Docelowa wieża
1	4	0	1
2	1	1	1
3	3	2	2
4	2	3	3
5	5	4	3

Stąd `rankOf[4] = 0`, `rankOf[2] = 3`, a `destination[2] = 3`.

5.2. Dlaczego rang jest dokładnie n?

Mamy n różnych krążków. Każdy występuje w konfiguracji docelowej dokładnie raz, więc możemy przypisać dokładnie jedną z rang $0, 1, \ldots, n-1$. Dzięki temu sortowanie według rang nie ma remisów.

5.3. Zebranie wszystkich krążków

Stan aktualny: `T1=[2,4,5]`, `T2=[1]`, `T3=[3]`.

Wykonujemy `2 → 1`, a potem `3 → 1`. Otrzymujemy:

```
T1 = [2, 4, 5, 1, 3]
T2 = []
T3 = []
```

Przepisując krążki na ich rangi, widzimy:

```
krążki: 2  4  5  1  3
rangi:  3  0  4  1  2
```

Teraz problem brzmi: „jak przy pomocy dwóch kolejek posortować `3,0,4,1,2`?”.

Lekcja 6. Stabilne rozdzielanie według jednego bitu

6.1. Co znaczy „stabilnie”?

Jeżeli sortowanie dzieli elementy na dwie kategorie, powinno zachować ich wzajemną kolejność **wewnątrz każdej kategorii**. To jest stabilność.

Na przykład dla rang `[3,0,4,1,2]` najmłodszy bit (bit 0) ma wartości:

```
rangi:     3   0   4   1   2
bit 0:     1   0   0   1   0
```

Najpierw chcemy otrzymać wszystkie elementy z bitem `0`, potem wszystkie z bitem `1`:

```
zera:      0   4   2
jedynki:   3   1
wynik:     0   4   2   3   1
```

Zauważ, że w grupie zer nadal mamy `0,4,2`, a w grupie jedynek `3,1`. Niczego nie odwracamy.

6.2. Jeden przebieg: dokładna symulacja

Początkowo `T1=[2,4,5,1,3]` (rangi `[3,0,4,1,2]`), `T2=[]`.

Reguła: jeżeli aktualny bit rangi jest `0`, wykonaj `1 → 1`; jeśli jest `1`, wykonaj `1 → 2`. Zrób to **dokładnie n razy**.

Krok	Czytana ranga	Bit 0	Ruch	T1 po ruchu (krążki)	T2 po ruchu
1	3	1	`1 2`	`[4,5,1,3]`	`[2]`
2	0	0	`1 1`	`[5,1,3,4]`	`[2]`
3	4	0	`1 1`	`[1,3,4,5]`	`[2]`
4	1	1	`1 2`	`[3,4,5]`	`[2,1]`
5	2	0	`1 1`	`[4,5,3]`	`[2,1]`

Zostały dwa krążki na wieży 2. Przenosimy je po kolei `2 → 1`, więc otrzymujemy:

```
T1 = [4,5,3,2,1]
T2 = []
ranki T1 = [0,4,2,3,1]
```

Dlaczego to działa? Zera krążą w T1 i pozostają tam w kolejności napotkania. Jedynki ustawiają się w T2 także w kolejności napotkania. Na końcu dokładamy T2 po zerach.

6.3. Dlaczego wykonujemy dokładnie n kroków?

Początkowo na T1 leży n krążków. Każda operacja usuwa *następny jeszcze nieprzetworzony krążek* z przodu. Krążek z bitem `1` idzie do T2. Krążek z bitem `0` wraca na koniec T1, **za wszystkie jeszcze nieprzetworzone krążki**. Dlatego przez pierwsze n operacji każdy początkowy krążek jest badany raz — nawet jeśli niektóre już wróciły do T1.

Po tych n krokach na T1 znajdują się wyłącznie „zera", a na T2 wyłącznie „jedynki".

> **Uwaga dla początkujących:** gdy napotkasz `1 → 1`, nie skracaj go do „nic nie rób". Taka zmiana zepsuje kolejność i cały algorytm.

Lekcja 7. Radix Sort: od najmłodszego bitu

7.1. Jeden bit nie wystarcza

Po poprzedniej lekcji rangi to `[0,4,2,3,1]`. Są poprawnie podzielone tylko według bitu 0. Musimy jeszcze przejrzeć bit 1 oraz bit 2.

LSD Radix Sort (Least Significant Digit) oznacza sortowanie od najmniej znaczącej cyfry — tutaj od najmłodszego bitu. Każdy przebieg musi być stabilny. Dzięki temu to, co wcześniej uporządkowaliśmy, pozostaje poprawne w obrębie następnych grup.

7.2. Pełny przykład z pięcioma rangami

Stan	Rangi na T1 od przodu do końca	Co właśnie uporządkowano?
Początek	`[3,0,4,1,2]`	jeszcze nic
Po bicie 0	`[0,4,2,3,1]`	ostatni bit
Po bicie 1	`[0,4,1,2,3]`	dwa ostatnie bity
Po bicie 2	`[0,1,2,3,4]`	wszystkie trzy bity

Powstała kolejność krążków odpowiadająca rangom `[0,1,2,3,4]` to:

```
T1 = [4, 1, 3, 2, 5]
```

Zwróć uwagę: **posortowane są rangi, nie numery krążków**. Porządek `4,1,3,2,5` jest dokładnie tym, czego potrzebujemy.

7.3. Rozdzielenie po sortowaniu

Odczytujemy pierwszy krążek T1 i przenosimy go na koniec jego wieży docelowej, dokładnie n razy:

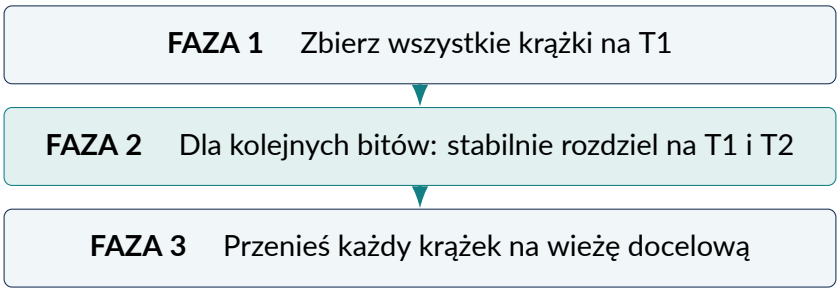
Krok	Krążek	Wieża docelowa	Ruch
1	4	1	`1 1`
2	1	1	`1 1`
3	3	2	`1 2`
4	2	3	`1 3`
5	5	3	`1 3`

Wynik: `T1=[4,1]`, `T2=[3]`, `T3=[2,5]` — dokładnie konfiguracja docelowa.

Dlaczego dołożenie na wieżę 1 jest bezpieczne? Krążki docelowo należące do T1 są już na początku całkowitego porządku rang. Przesuwamy je na koniec bieżącej kolejki T1 i nie spotykamy

ich ponownie, zanim nie przetworzymy wszystkich n pierwotnych pozycji.

Lekcja 8. Trzy fazy jako jeden algorytm



Pseudokod rozwiązania:

```
Wczytaj stan docelowy.
Dla każdego krążka zapisz rangę i docelową wieżę.
Wczytaj stan aktualny.

Dla każdej wieży 2..k:
    dopóki wieża nie jest pusta:
        przenieś jej pierwszy krążek na koniec wieży 1

Dla bit = 0, 1, ..., B-1:
    powtórz n razy:
        sprawdź bit rangi pierwszego krążka z wieży 1
        jeśli bit = 0: przenieś 1 -> 1
        jeśli bit = 1: przenieś 1 -> 2
    dopóki wieża 2 nie jest pusta:
        przenieś 2 -> 1

Powtórz n razy:
    weź pierwszy krążek z wieży 1
    przenieś go z 1 na jego wieżę docelową
```

Najważniejszy element myślenia konstrukcyjnego: przekształcamy dowolną konfigurację w **stan kanoniczny** (jedna uporządkowana kolejka), a z niego budujemy właściwy wynik.

Lekcja 9. Dowód poprawności krok po kroku

9.1. Co to jest dowód poprawności?

Test na 10, 100 albo milionie przypadków może wykryć błąd, ale nie dowiedzie, że algorytm działa na każdym dopuszczalnym wejściu. Dowód wykorzystuje własności, które są zawsze prawdziwe.

9.2. Lemat 1: poprawność pojedynczego rozdzielenia

Teza. Jeden przebieg po danym bicie tworzy na T1 stabilnie połączone grupy zer i jedynek.

Dlaczego? W dokładnie n krokach każdy rozpatrywany krążek trafia: z bitem `0` na koniec T1, z bitem `1` na koniec T2. W obu grupach zachowujemy kolejność napotkania. Po przeniesieniu zawartości T2 na koniec T1 cała kolejka składa się najpierw z zer, potem z jedynek, z zachowaniem porządku wewnątrz grup. Koniec dowodu.

9.3. Lemat 2: poprawność sortowania

Teza. Po zakończeniu przebiegu dla bitu i rangi są uporządkowane według $i+1$ najmłodszych bitów.

Początek: dla bitu `0` jest to po prostu poprzedni lemat.

Krok: załóżmy, że porządek według młodszych bitów już obowiązuje. Stabilny podział po kolejnym bicie stawia zera przed jedynkami, ale w każdej grupie zachowuje wcześniejszy porządek. Zatem całe `i+1` młodszych bitów jest uporządkowane.

Wniosek: po B przebiegach porządek według wszystkich bitów jest poprawny. Ponieważ rangi są różne, kolejka ma dokładnie kolejność `0,1,...,n-1`.

9.4. Lemat 3: poprawność końcowego rozdzielenia

Rangi uporządkowano w kolejności zgodnej z przejściem po docelowych wieżach od 1 do k, w każdej od góry do dołu. Gdy przenosimy krążki na spód ich wież, trafiają tam w tej samej kolejności, co w opisie stanu docelowego. Na końcu każda wieża ma wymaganą zawartość.

Z lematów 1–3 wynika, że cały algorytm jest poprawny. Dodatkowo każda operacja zawsze zdejmuje element z niepustej kolejki (w przebiegu dokładnie n nieprzetworzonych krążków jest dostępnych, a pętle opróżniające sprawdzają niepustość).

Lekcja 10. Liczenie ruchów i pamięci

10.1. Ile ruchów ma pierwsza faza?

Niech d_1 oznacza liczbę krążków na pierwszej wieży **w stanie aktualnym**. Na innych wieżach leży $n - d_1$ krążków; każdy przenosimy raz. Wykonujemy więc dokładnie:

$$M_1 = n - d_1.$$

10.2. Ile ruchów ma jeden przebieg?

Dla bitu i:

- dokładnie n ruchów przetwarzających kolejkę T1;
- dodatkowo po jednym ruchu dla każdego krążka, którego i-ty bit rangi wynosi `1` (powrót z T2 na T1).

Po wszystkich B bitach:

$$M_2 = nB + \sum_{r=0}^{n-1} \text{popcount}(r).$$

Funkcja $\text{popcount}(r)$ zlicza jedynki w zapisie binarnym liczby r. Na przykład $\text{popcount}(5) = 2$, bo `5 = 101`$_2$.

Dlaczego występuje suma? Każdy bit `1` w każdej randze powoduje jeden dodatkowy ruch powrotny w odpowiednim przebiegu.

10.3. Trzecia faza i pełny wzór

Trzecia faza wykonuje dokładnie n ruchów, także wtedy, gdy docelowa wieża to wieża 1. Stąd:

$$\boxed{M = (n - d_1) + nB + \sum_{r=0}^{n-1} \text{popcount}(r) + n.}$$

Wzór pozwala wypisać liczbę ruchów **przed** ich wykonaniem. Nie trzeba przechowywać milionów par `(a,b)` w pamięci.

10.4. Dlaczego limit 4 milionów jest spełniony?

Każda ranga r ma najwyżej B bitów, więc $\text{popcount}(r) \le B$. Stąd:

$$M \le n + nB + nB + n = 2n(B + 1).$$

Ponieważ $B \le 17$ oraz $n \le 100\,000$, otrzymujemy:

$$M \le 2 \cdot 100\,000 \cdot 18 = 3\,600\,000 < 4\,000\,000.$$

To **formalna gwarancja**, niezależna od konfiguracji wejściowej.

Możemy też policzyć dokładniejszą granicę dla $n = 100\,000$. Suma liczby jedynek we wszystkich rangach `0..99999` wynosi **815 024**. Przy pustej początkowo T1:

$$M = 100\,000 + 1\,700\,000 + 815\,024 + 100\,000 = 2\,715\,024.$$

To jeszcze większy zapas. W praktyce nie musisz wyprowadzać liczby 815 024 ręcznie — wystarczy prosty przebieg po rangach w programie.

10.5. Złożoność obliczeniowa

Złożoność czasowa opisuje, jak liczba operacji rośnie wraz z n. Mamy $B = O(\log n)$ przebiegów po n elementach. Otrzymujemy:

$$\boxed{O(n \log n + k) \text{ czasu}, \qquad O(n + k) \text{ pamięci}.}$$

Symbol $O(\cdot)$ opisuje rząd wzrostu, a nie dokładną liczbę ruchów. Przykładowo $O(n \log n)$ oznacza, że liczba działań rośnie mniej więcej proporcjonalnie do $n \log n$, pomijając stałe mnożniki.

Przechowujemy każdy krążek na dokładnie jednej z k wież; dodatkowo dwie tablice informacji o krążkach. To wyraźnie mniej niż 256 MB dla $n \le 100\,000$.

Lekcja 11. Pełny program w C++17

Kod jest gotowym plikiem `hak.cpp`. Używa wyłącznie standardowej biblioteki C++ i wypisuje instrukcję w formacie wymaganym przez zadanie.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    int k;
    if (!(cin >> k)) {
        return 0;
    }

    vector<vector<int>> target(k);
    int n = 0;

    // Read the target configuration.
    for (int i = 0; i < k; ++i) {
        int c;
        cin >> c;
        target[i].resize(c);
        n += c;

        for (int &disk : target[i]) {
            cin >> disk;
        }
    }

    // Assign a unique target rank to each disk.
    vector<int> rankOf(n + 1);
    vector<int> destination(n + 1);

    int rank = 0;
    for (int i = 0; i < k; ++i) {
        for (int disk : target[i]) {
            rankOf[disk] = rank++;
            destination[disk] = i;
        }
    }

    // Each tower is represented as a FIFO queue.
    vector<deque<int>> towers(k);

    // Read the current configuration.
    for (int i = 0; i < k; ++i) {
        int d;
        cin >> d;
        for (int j = 0; j < d; ++j) {
            int disk;
            cin >> disk;
            towers[i].push_back(disk);
        }
    }

    // Number of bits required for ranks 0..n-1.
    int bits = 0;
    while ((1 << bits) < n) {
        ++bits;
    }

    // Calculate the exact number of moves.
    long long numberOfMoves =
        n - static_cast<int>(towers[0].size());

    numberOfMoves += 1LL * n * bits;
    numberOfMoves += n;
```

```
    for (int r = 0; r < n; ++r) {
        numberOfMoves +=
            __builtin_popcount(static_cast<unsigned>(r));
    }

    cout << numberOfMoves << '\n';

    // Perform and immediately print a legal move.
    auto moveDisk = [&](int from, int to) {
        int disk = towers[from].front();
        towers[from].pop_front();
        towers[to].push_back(disk);

        cout << from + 1 << ' '
             << to + 1 << '\n';
    };

    // Phase 1: Gather all disks on tower 1.
    for (int i = 1; i < k; ++i) {
        while (!towers[i].empty()) {
            moveDisk(i, 0);
        }
    }

    // Phase 2: Stable LSD radix sort.
    for (int bit = 0; bit < bits; ++bit) {
        for (int j = 0; j < n; ++j) {
            int disk = towers[0].front();

            if ((rankOf[disk] >> bit) & 1) {
                moveDisk(0, 1);
            } else {
                moveDisk(0, 0);
            }
        }

        while (!towers[1].empty()) {
            moveDisk(1, 0);
        }
    }

    // Phase 3: Distribute disks to target towers.
    for (int j = 0; j < n; ++j) {
        int disk = towers[0].front();
        moveDisk(0, destination[disk]);
    }

    return 0;
}
```

Lekcja 12. Czytamy kod, zamiast go przepisywać

12.1. `vector` i `deque`

`vector<int>` to tablica, której długość może być ustalana podczas działania programu. `vector<vector<int>> target(k)` tworzy `k` osobnych tablic — po jednej na wieżę docelową. `deque<int>` jest kolejką umożliwiającą szybkie operacje z obu końców.

Instrukcja C++	Znaczenie
`cin >> x`	wczytaj liczbę `x`
`cout << x`	wypisz `x`
`v.size()`	liczba elementów tablicy / kolejki
`q.front()`	odczytaj pierwszy element, nie usuwając go
`q.pop_front()`	usuń pierwszy element
`q.push_back(x)`	dopisz `x` na koniec
`q.empty()`	sprawdź, czy kolejka jest pusta
`i++`	zwiększ `i` o 1

W C++ pozycje w tablicach liczymy od `0`, natomiast w treści zadania wieże od `1`. Dlatego wewnętrznie używamy indeksów `0..k-1`, a wypisujemy `from + 1` i `to + 1`.

12.2. Pętle i warunki

Pętla `for (int i = 0; i < k; ++i)` wykonuje instrukcję kolejno dla `i=0,1,...,k-1`. Pętla `while (!q.empty())` działa tak długo, jak `q` nie jest pusta. Warunek `if` wybiera jedną z dwóch możliwości.

12.3. Wyciąganie pojedynczego bitu

Najbardziej tajemnicza instrukcja:

```
if ((rankOf[disk] >> bit) & 1)
```

Rozłóżmy ją na części. `>> bit` przesuwa zapis binarny w prawo o `bit` miejsc. `& 1` zostawia tylko najmłodszy bit. Na przykład dla rangi `5` mamy 101_2:

```
5 >> 0  = 101₂   ;  (101₂ & 1) = 1
5 >> 1  =  10₂   ;  ( 10₂ & 1) = 0
5 >> 2  =   1₂   ;  (  1₂ & 1) = 1
```

Wynik wynosi więc `1`, `0`, `1` dla kolejnych bitów `0`, `1`, `2`.

12.4. Funkcja wykonująca ruch

`auto moveDisk = [&](int from, int to) { ... };` tworzy krótką funkcję (wyrażenie lambda), która ma dostęp do kolejek `towers`. W jej wnętrzu:

1. Odczytujemy krążek z `towers[from].front()`.
2. Usuwamy go z początku `pop_front()`.
3. Dokładamy na koniec `towers[to].push_back(disk)`.
4. Wypisujemy ruch przy indeksach wież powiększonych o 1.

W ten sposób reguła gry jest zaimplementowana **w jednym miejscu**. Jeżeli trzeba poprawić sposób zapisu operacji, nie trzeba poprawiać wielu fragmentów kodu.

12.5. Po co `long long` i `1LL`?

`long long` przechowuje duże liczby całkowite. `1LL * n * bits` wymusza obliczanie iloczynu w typie `long long`. Tutaj nawet 32-bitowy `int` wystarczyłby do limitu ruchów, ale nawyk dobierania bezpiecznego typu jest bardzo przydatny w konkursach.

12.6. Dlaczego program nie zapisuje wszystkich ruchów?

Pierwsza linia wyjścia musi zawierać liczbę ruchów. Obliczamy ją wzorem z lekcji 10, po czym każdy ruch wypisujemy od razu. Dzięki temu nie potrzebujemy tablicy o milionach par liczb.

Wskazówka: do sprawdzania programu lokalnie można przekierować wyjście do pliku. Dla dużych przypadków terminal nie jest dobrym miejscem do wyświetlania milionów linijek.

Lekcja 13. Jak samodzielnie testować rozwiązania

13.1. Kompilacja

Zapisz cały kod w pliku `hak.cpp`. W terminalu wpisz:

```
g++ -std=c++17 -O2 -Wall -Wextra hak.cpp -o hak
```

Utwórz plik `test.txt` z przykładem z lekcji 4, a następnie uruchom:

```
./hak < test.txt > wynik.txt
```

W pierwszym wierszu `wynik.txt` powinna znajdować się liczba wypisanych następnie ruchów. Dla naszego przykładu jest to **27**:

- zebranie: 2 ruchy;
- sortowanie: $5 \cdot 3 + (0 + 1 + 1 + 2 + 1) = 20$ ruchów;
- rozdzielenie: 5 ruchów.

Sprawdzenie rachunku: rangi to `0,1,2,3,4`, a liczby jedynek w ich zapisach binarnych to `0,1,1,2,1`, suma `5`. Razem `2 + 15 + 5 + 5 = 27` ruchów. Warto zawsze weryfikować sumę składników przed wypisaniem wyniku.

13.2. Testy brzegowe

Sprawdź przypadki, w których:

- jest tylko jeden krążek (`n=1`) — nie ma przebiegów sortowania;
- pierwsza wieża jest pusta;
- wszystkie krążki już leżą w docelowych miejscach;
- są tylko dwie wieże (`k=2`);
- są aż pięć wież (`k=5`);
- wszystkie krążki mają docelowo trafić na pierwszą wieżę;
- w danym przebiegu wszystkie bity wynoszą `0` albo wszystkie wynoszą `1`.

Nie zakładaj, że jeśli stan jest już poprawny, to program musi wypisać `0`. W tym zadaniu dodatkowe legalne ruchy są dopuszczalne, o ile wynik się zgadza, a limit nie jest przekroczony.

13.3. Symulator jako niezależny kontroler

W zadaniach konstrukcyjnych najskuteczniejszym testem jest drugi, krótki program, który:

1. Wczytuje stan aktualny.
2. Odczytuje liczbę `m` i dokładnie `m` ruchów z wyniku rozwiązania.
3. Sprawdza, czy źródłowa wieża jest niepusta i numery wież są poprawne.
4. Stosuje każdy ruch dosłownie według reguł gry.
5. Porównuje otrzymaną konfigurację ze stanem docelowym.

Symulator powinien być jak najprostszy i możliwie niezależny od sprawdzanego algorytmu. Wtedy łatwiej wykryć w nim błędy.

13.4. Weryfikacja tej implementacji

Przed przygotowaniem tego podręcznika kod C++17 został skompilowany i przetestowany niezależną symulacją na **330 losowo wygenerowanych przykładach** obejmujących $k = 2..5$ oraz różne

liczby krążków. Sprawdzono legalność każdego ruchu, zgodność ich deklarowanej liczby z wypisanymi operacjami i końcowy układ wież. Wszystkie te testy zakończyły się poprawnie. Nie jest to dowód poprawności dla wszystkich wejść; dowód podano w lekcji 9.

Lekcja 14. Najczęstsze błędy i jak ich unikać

Błąd	Dlaczego jest niebezpieczny?	Jak poprawić?
Traktowanie `1 → 1` jako braku ruchu	Kolejka się nie obraca; sortowanie jest błędne.	Zawsze wykonuj `pop_front` i `push_back`.
Sortowanie według numerów krążków	Numer krążka nie określa miejsca docelowego.	Najpierw wyznacz rangi `0..n-1`.
Podział według bitów od najstarszego	Prosta wersja tego algorytmu wymaga porządku LSD.	Zacznij od bitu `0`.
Niestabilny podział	Kolejny bit niszczy porządek młodszych bitów.	Używaj kolejek, zachowaj kolejność dopisywania.
Brak `n` ruchów w końcowej fazie	Wygodne skracanie ruchów na T1 psuje wzór na `m`.	Wykonuj dokładnie po jednej operacji na krążek.
Wypisywanie indeksów od zera	W zadaniu wieże mają numery od `1`.	Dodaj `1` przy wypisywaniu.
Przechowywanie wszystkich ruchów	Niepotrzebne zużycie pamięci.	Najpierw policz `m`, potem wypisuj na bieżąco.
Pomylenie stanu docelowego z aktualnym	Program odtwarza niewłaściwą konfigurację.	Czytaj najpierw docelową, potem aktualną.

Lekcja 15. Ćwiczenia do samodzielnego rozwiązania

Najpierw spróbuj bez patrzenia na odpowiedzi. Przy każdym ćwiczeniu zapisz **nie tylko wynik, lecz także sposób rozumowania**.

Poziom 1 — podstawy

Ćwiczenie 1. Na wieżach jest kolejno `3`, `0`, `5`, `2` krążków. Oblicz k oraz n, a następnie zapisz liczbę krążków jako sumę.

Ćwiczenie 2. Podaj wartości 2^4, 2^7 i 2^{10}. Ile bitów potrzeba dla $n = 9$ różnych rang?

Ćwiczenie 3. Dla kolejki `[8,1,6,3]` wykonaj trzy obroty `1 → 1`. Podaj kolejkę po każdym ruchu.

Ćwiczenie 4. Zapisz liczby `6`, `9`, `13` w systemie binarnym. Zaznacz najmłodszy bit każdej z nich.

Poziom 2 — modelowanie i sortowanie

Ćwiczenie 5. Stan docelowy: `T1=[9,3]`, `T2=[]`, `T3=[7,4,1]`. Utwórz tabelę `krążek → ranga → wieża docelowa`.

Ćwiczenie 6. Rangi w kolejce to `[5,2,7,0,3]`. Rozdziel stabilnie według bitu `0`. Zapisz osobno listę zer i jedynek, a potem wynik.

Ćwiczenie 7. Rangi `[3,0,2,1]` posortuj od początku do końca LSD Radix Sort. Zapisz stan po bicie `0` i po bicie `1`.

Ćwiczenie 8. Wśród rang `0..7` policz sumę `popcount`. Ile powrotnych ruchów `2 → 1` wystąpi łącznie podczas wszystkich przebiegów?

Poziom 3 — dowodzenie i analiza

Ćwiczenie 9. Wyjaśnij jednym akapitem, dlaczego podczas jednego przebiegu nie możemy przetworzyć tego samego krążka dwa razy w pierwszych n krokach.

Ćwiczenie 10. Dla $n = 8, k = 2, d_1 = 3$ oblicz dokładną liczbę ruchów algorytmu ze wzoru.

Ćwiczenie 11. Czy w końcowej fazie wolno zastąpić `1 → 1` nicnierobieniem, **jeśli jednocześnie poprawimy wzór na liczbę ruchów**? Zastanów się, czy zostanie zachowana kolejność. Odpowiedz z krótkim uzasadnieniem.

Ćwiczenie 12. Zaproponuj trzy przypadki brzegowe, które chciałbyś dodać do automatycznych testów, i napisz, jakiego błędu każdy szuka.

Rozwiązania ćwiczeń

A1. $3 \cdot 4 + 2 = 14$. **A2.** $2^6 = 64$. **A3.** Trzy bity (rangi `0..5`). **A4.** 1101_2, bo $13 = 8 + 4 + 1$.

K1. `[4,1,3,2] → [1,3,2,4] → [3,2,4,1]`. **K2.** Nie; zawsze wolno zdjąć tylko górny krążek.

1. $k = 4, n = 3 + 0 + 5 + 2 = 10$.

2. $2^4 = 16, 2^7 = 128, 2^{10} = 1024$. Dla $n = 9$ potrzeba $B = 4$, ponieważ $2^3 < 9 \le 2^4$.

3. `[1,6,3,8]`, potem `[6,3,8,1]`, potem `[3,8,1,6]`.

4. $6 = 110_2$ (bit 0: `0`), $9 = 1001_2$ (bit 0: `1`), $13 = 1101_2$ (bit 0: `1`).

5. `9→0→T1`, `3→1→T1`, `7→2→T3`, `4→3→T3`, `1→4→T3`. T2 pozostaje pusta.

6. Zera `[2,0]`, jedynki `[5,7,3]`; wynik `[2,0,5,7,3]`.

7. Bit 0: zera `[0,2]`, jedynki `[3,1]`, wynik `[0,2,3,1]`. Bit 1: zera `[0,1]`, jedynki `[2,3]`, wynik `[0,1,2,3]`.

8. `popcount(0..7)` to `0+1+1+2+1+2+2+3=12`. Jest zatem 12 powrotów `2 → 1`.

9. W chwili przesuwania krążka z bitem `0` na koniec T1 stoi on za wszystkimi jeszcze nieprzetworzonymi. Krążek z bitem `1` opuszcza T1. Dlatego przed upływem n operacji na początek T1 nie może wrócić wcześniej przetworzony krążek.

10. $B = 3$, suma `popcount(0..7)=12`, więc $M = (8 - 3) + 8 \cdot 3 + 12 + 8 = 49$.

11. Nie w sposób polegający wyłącznie na pomijaniu takich operacji. Jeżeli pierwszy krążek należy docelowo do T1, to pozostawiony na początku utrudni przejście do następnego, nieprzetworzonego krążka. Ruch `1 → 1` jest potrzebny do przejścia dalej. Optymalizowanie sekwencji wymagałoby innego, poprawnie uzasadnionego algorytmu.

12. Przykładowo: `n=1` wykrywa błędy przy `bits=0`; pusta T1 wykrywa błędy zbierania; już poprawny stan wykrywa fałszywe założenie, że program nie może wykonywać dodatkowych ruchów.

Lekcja 16. Jak samodzielnie wymyślać rozwiązania podobnych zadań?

Gdy napotkasz następne zadanie konstrukcyjne, nie zaczynaj od kodowania. Najpierw przejdź przez poniższą procedurę. Dzięki niej rozpoznasz, jakiego rodzaju rozwiązania szukać.

Krok 1. Przetłumacz treść na operacje

Wypisz, co wolno zrobić w **jednym** ruchu. Spróbuj zilustrować tę operację na trzech elementach. W HAK operacją nie jest „przełóż dowolny krążek”, tylko „zdejmij pierwszy i dołóż na koniec”. To prowadzi do kolejki, a nie do stosu.

Krok 2. Oddziel wymagany wynik od sposobu jego osiągnięcia

Czy musisz uzyskać rozwiązanie najkrótsze, czy wystarczy jakiekolwiek poprawne? Czy zadanie wymaga konkretnego przebiegu, czy jedynie konfiguracji końcowej? W HAK nie szukamy tego, co robił Bajtek. Budujemy własną instrukcję.

Krok 3. Zamień limit na budżet na element

Tutaj limit wynosi 4 000 000, a krążków może być 100 000. Dzieląc pierwszą liczbę przez drugą, otrzymujemy **40 ruchów na krążek średnio**. To cenna wskazówka: algorytm wykonujący po kilkanaście przebiegów po wszystkich elementach może się zmieścić, ale algorytm wykonujący osobno tysiące ruchów na każdy element już nie.

Krok 4. Poszukaj stanu pośredniego

Zamiast bezpośrednio budować trudny stan, spróbuj osiągnąć prosty stan *kanoniczny*, który umiesz kontrolować. W HAK jest nim jedna kolejka z krążkami uporządkowanymi według rang. W innych zadaniach może nim być pusta plansza, posortowana tablica albo wspólny punkt startowy.

Krok 5. Zmień nazwy elementów, jeśli upraszczają problem

Oryginalne numery krążków tylko utrudniają myślenie. Rangi zamieniają dowolny cel na zwykłą kolejność `0,1,...,n-1`. Bardzo często najważniejszą częścią rozwiązania jest dobranie nowej reprezentacji, a nie skomplikowana implementacja.

Krok 6. Znajdź małą operację, którą możesz wielokrotnie powtarzać

W HAK odkryliśmy stabilny podział według pojedynczego bitu. Nie sortuje on od razu wszystkiego, ale jest prosty, legalny i zachowuje ważną własność. Kilka takich podziałów daje pełne sortowanie.

Krok 7. Zapisz niezmiennik i oszacuj koszt przed programowaniem

Najpierw odpowiedz: „co dokładnie jest prawdziwe po jednym przebiegu?”. Dopiero potem: „ile razy go wykonam i ile ruchów to kosztuje?”. Właśnie tak powstaje dowód, a nie dopiero po napisaniu kodu.

Krok 8. Rozdziel generator od kontrolera

Program główny ma generować kroki. Osobny symulator powinien weryfikować, czy te kroki są legalne i prowadzą do celu. To uniwersalny schemat dla zadań z konstruowaniem ścieżek, operacji na tablicach, permutacji i transformacji stanów.

Pytanie do samodzielnego przemyślenia: co byłoby trudniejsze, gdyby zabroniono ruchów z wieży na tę samą wieżę? Wtedy nie mielibyśmy darmowej w sensie prostoty operacji obrotu T1, na której opiera się stabilny podział. Dlatego nawet z pozoru drobny szczegół zasad może całkowicie zmienić algorytm.

Cztery tygodnie nauki od zera

Nie ucz się jednocześnie całej matematyki i całego C++. Buduj kompetencje warstwami. Przy codziennej nauce przez 45–60 minut proponuję taki plan:

Tydzień	Temat	Sprawdzian na koniec tygodnia
1	Zmienne, działania, potęgi, tablice, pętle	Obliczasz n, B, rozumiesz tablice i `for`.
2	Kolejki, symulacja, warunki, funkcje	Umiesz napisać `moveDisk` bez kopiowania.
3	Bity, stabilność, sortowanie Radix Sort	Ręcznie sortujesz pięć rang w trzech przebiegach.
4	Niezmienniki, dowody, testy, złożoność	Rozumiesz i potrafisz wyjaśnić cały algorytm HAK.

Co rozwiązywać dalej?

Po tym zadaniu przećwicz małe problemy z kolejkami i symulacją, następnie sortowanie przez zliczanie i Radix Sort, a dopiero później zadania o mocniejszych niezmiennikach i zaawansowanych strukturach danych. Lepsze jest pełne zrozumienie jednego rozwiązania niż przepisanie dziesięciu trudnych programów.

Jedna strona do powtórki

Model: wieża to kolejka FIFO. Operacja `a → b` usuwa element z początku `a` i dopisuje go na koniec `b`. Operacja `a → a` to obrót kolejki.

Cel: dowolna legalna instrukcja licząca co najwyżej 4 000 000 ruchów; nie trzeba minimalizować liczby ruchów.

Trzy fazy: zbierz na T1 → posortuj rangi w dwóch kolejkach → rozdziel według wież docelowych.

Rangi: czytaj docelowe wieże po kolei, od góry do dołu, i nadaj rangi `0..n-1`.

Jeden bit: `0` powoduje `1 → 1`, `1` powoduje `1 → 2`, następnie wszystkie krążki `2 → 1`. Rozdzielenie jest stabilne.

Kolejność bitów: od najmłodszego do najstarszego (LSD).

Liczba bitów: $B = \lceil \log_2 n \rceil$, dla $n = 1$ przyjmujemy `0`.

Dokładna liczba ruchów:

$$M = (n - d_1) + nB + \sum_{r=0}^{n-1} \text{popcount}(r) + n.$$

Bezpieczna granica: $M \le 2n(B+1) \le 3\,600\,000$.

Złożoność: $O(n \log n + k)$ czasu oraz $O(n + k)$ pamięci.

Pytanie kontrolne: czy potrafisz wyjaśnić, dlaczego w pojedynczym przebiegu obrotu i rozdzielenia każdy pierwotny krążek jest obsłużony dokładnie raz? Jeśli tak, rozumiesz najważniejszy pomysł tego rozwiązania.